Project-1

1. Problem Statement

Develop a Python program to calculate a student's final marks by applying grace marks to subjects in which the student scores less than 35 marks. The program should calculate the total marks, display the grace marks used, and determine the student's result and class based on the final marks.

2. Objectives

1. To accept the number of subjects, grace marks available, and marks obtained in each subject.
2. To apply grace marks to subjects with marks below 35, provided the available grace marks are sufficient.
3. To calculate and display the final marks and total marks.
4. To determine whether the student has passed or failed.
5. To classify the student's result as Distinction, First Class, Second Class, or Pass.

3. Assumptions

1. The number of subjects and grace marks are entered as integers.
2. Marks for all subjects are entered as integers separated by spaces.
3. The passing mark for each subject is 35.
4. Grace marks are applied only when the required marks are within the available grace marks.
5. The result is determined using the final marks after applying grace marks.

6. The program assumes that the number of marks entered matches the number of subjects.

Grace Marks — Pseudocode & Flowchart

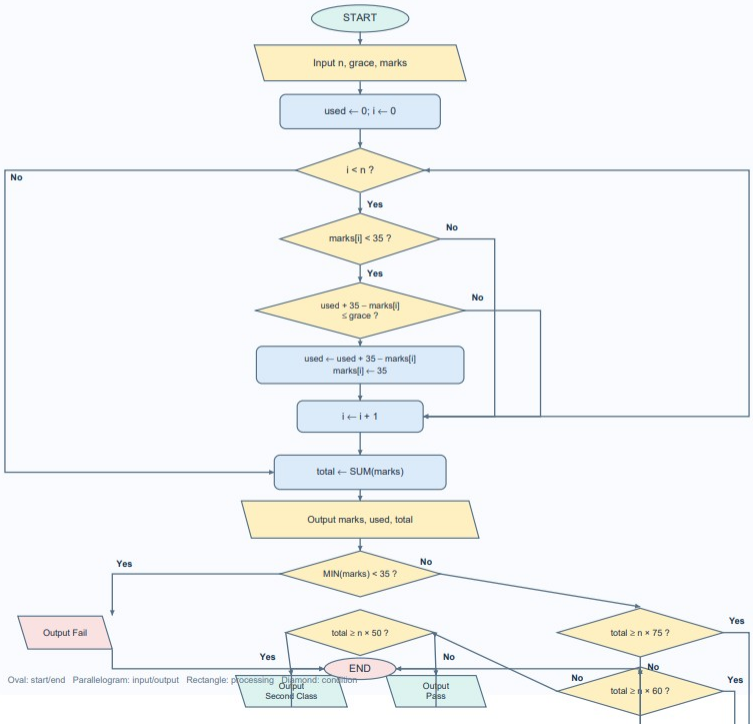
Logic mapped directly from pp-1.py

1. Pseudocode

```
START
INPUT n (number of subjects)
INPUT grace (available grace marks)
INPUT marks list
used ← 0
FOR i ← 0 TO n – 1
  IF marks[i] < 35 AND
     used + 35 – marks[i] ≤ grace
    used ← used + 35 – marks[i]
    marks[i] ← 35
  END IF
END FOR
total ← SUM(marks)
OUTPUT marks, used, total
IF MIN(marks) < 35
  OUTPUT "Fail"
ELSE IF total ≥ n × 75
  OUTPUT "Distinction"
ELSE IF total ≥ n × 60
  OUTPUT "First Class"
ELSE IF total ≥ n × 50
  OUTPUT "Second Class"
ELSE
  OUTPUT "Pass"
END IF
END
```

Assumes the input marks list contains at least n values.

2. Flowchart



Algorithm

1. Start.
2. Enter the number of subjects and maximum grace marks.
3. Enter the marks for each subject.
4. Check whether the marks are valid.
5. Identify the subjects in which the student failed.
6. Apply grace marks if allowed.
7. Calculate the total marks.
8. Check whether the student passed or failed.
9. Determine the result classification.
10. Display the final marks, grace marks used, total marks, and result.
11. Stop.

Code

```
n = int(input("Enter subjects: "))
grace = int(input("Enter grace marks: "))
marks = list(map(int, input("Enter marks separated by spaces: ").split()))
used = 0
for i in range(n):
    if marks[i] < 35 and used + 35 - marks[i] <= grace:
        used += 35 - marks[i]
        marks[i] = 35
total = sum(marks)
print("Final Marks:", marks)
print("Grace Used:", used)
print("Total:", total)
if min(marks) < 35: print("Fail")
elif total >= n*75: print("Distinction")
elif total >= n*60: print("First Class")
elif total >= n*50: print("Second Class")
else: print("Pass")
```

4. Output (Sample Output)

Enter subjects: 5

Enter grace marks: 10

Enter marks separated by spaces: 30 65 80 55 70

Final Marks: [35, 65, 80, 55, 70]

Grace Used: 5

Total: 305

First Class

5. Conclusion

The Python program successfully calculates final marks by applying available grace marks to subjects with marks below 35. It displays the final marks, grace marks used, and total marks, and determines the student's result and class according to the specified criteria.

Project-2

1. Problem Statement

Develop a Python program to calculate a student's attendance percentage and determine the number of additional classes the student must attend to reach the required attendance percentage. The program should also display the projected attendance percentage after attending 5, 10, 20, and 30 additional classes.

2. Objectives

1. To accept the number of classes attended, total classes conducted, and required attendance percentage.
2. To validate the attendance details entered by the user.
3. To calculate and display the current attendance percentage.
4. To determine the minimum number of additional classes required to reach the target attendance percentage.

5. To calculate the projected attendance percentage after attending 5, 10, 20, and 30 additional classes.

3. Assumptions

1. The number of classes attended and total classes are entered as integers.
2. The required attendance percentage is entered as a floating-point number.
3. The total number of classes must be greater than zero.
4. The number of classes attended must be between zero and the total number of classes.
5. Each additional class attended increases both the number of classes attended and the total number of classes by one.
6. The required attendance percentage is assumed to be a valid target for the calculation.

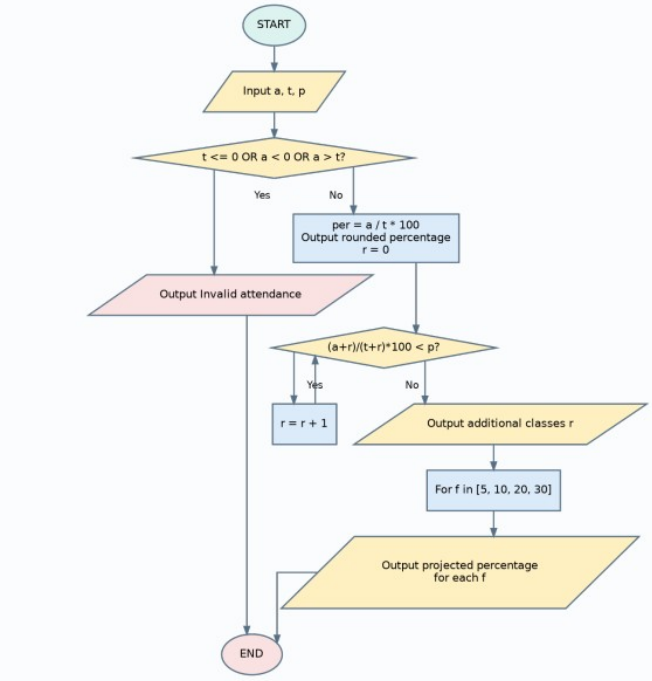
Attendance Percentage & Additional Classes

Source: pp-2.py

1. Pseudocode

```
START
INPUT attended a, total classes t, required % p
IF t <= 0 OR a < 0 OR a > t:
OUTPUT 'Invalid attendance'
ELSE:
per = a / t * 100
OUTPUT rounded attendance percentage
r = 0
WHILE (a+r)/(t+r)*100 < p:
r = r + 1
OUTPUT additional classes r
FOR f IN [5, 10, 20, 30]:
OUTPUT rounded (a+f)/(t+f)*100
END
```

2. Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Algorithm

1. Start.
2. Enter classes attended, total classes, and required attendance percentage.
3. Check whether the attendance details are valid.
4. Calculate the current attendance percentage.
5. Check whether the student meets the required attendance.
6. If eligible, display that no additional classes are required.
7. Otherwise, calculate the minimum additional classes required.
8. Display future attendance percentages.
9. Stop.

code

```
a = int(input("Classes attended: "))
t = int(input("Total classes: "))
p = float(input("Required percentage: "))
if t <= 0 or a < 0 or a > t:
    print("Invalid attendance")
else:
    per = a / t * 100
    print("Attendance:", round(per, 2), "%")
    r = 0
    while (a + r) / (t + r) * 100 < p:
        r += 1
    print("Additional classes:", r)
    for f in [5, 10, 20, 30]:
        print("After", f, "classes:", round((a + f) / (t + f) * 100, 2), "%")
```

4. Output (Sample Output)

Classes attended: 60

Total classes: 80

Required percentage: 75

Attendance: 75.0 %

Additional classes: 0

After 5 classes: 75.29 %

After 10 classes: 75.56 %

After 20 classes: 76.0 %

After 30 classes: 76.36 %

5. Conclusion

The Python program calculates the current attendance percentage and determines how many additional classes are needed to achieve the required attendance percentage. It also displays projected attendance percentages for different numbers of additional classes, helping students plan their attendance effectively.

Project-3

1. Problem Statement

Develop a Python program to process bank transactions, including deposits and withdrawals. The program should validate each transaction, ensure withdrawals do not exceed the specified withdrawal limit or available balance, and maintain the account balance. Finally, it should display the total deposits, total withdrawals, number of successful and failed transactions, and final account balance.

2. Objectives

1. To accept the initial account balance, number of transactions, and withdrawal limit.
2. To process deposit and withdrawal transactions.
3. To validate transaction types and amounts.
4. To prevent withdrawals that exceed the withdrawal limit or available balance.

5. To count successful and failed transactions.
6. To calculate and display the total deposits, total withdrawals, and final account balance.

3. Assumptions

1. The initial balance and withdrawal limit are entered as floating-point numbers.
2. The number of transactions is entered as an integer.
3. Transaction type 1 represents a deposit, and transaction type 2 represents a withdrawal.
4. Transaction amounts must be greater than zero.
5. A withdrawal is successful only if its amount does not exceed the withdrawal limit or the available balance.
6. Invalid transaction types and amounts are counted as failed transactions.
7. The program assumes that the initial balance and withdrawal limit are valid non-negative values.

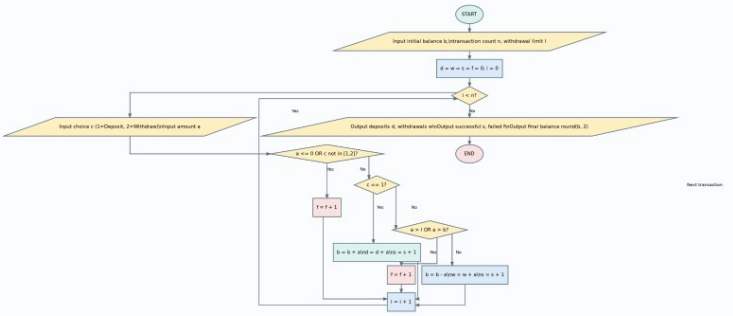
Bank Transaction Processing

Source: pp-3.py

1. Pseudocode

```
START
INPUT initial balance b
INPUT transaction count n
INPUT withdrawal limit l
d = w = s = f = 0
FOR i from 0 to n-1:
  INPUT choice c and amount a
  IF a <= 0 OR c not in [1, 2]:
    f = f + 1
  ELSE IF c == 1:
    b = b + a
    d = d + a; s = s + 1
  ELSE IF a > l OR a > b:
    f = f + 1
  ELSE:  # valid withdrawal
    b = b - a
    w = w + a; s = s + 1
END FOR
OUTPUT deposits d, withdrawals w
OUTPUT successful s, failed f
OUTPUT final balance round(b, 2)
END
```

2. Flowchart



Algorithm

1. Start.
2. Enter the initial balance, number of transactions, and withdrawal limit.
3. Enter the transaction type and amount.
4. Check whether the amount is valid.
5. If it is a deposit, add the amount to the balance.
6. If it is a withdrawal, check the withdrawal limit and available balance.
7. Update the balance and transaction counts.
8. Repeat for all transactions.
9. Display the transaction summary and final balance.
10. Stop.

code

```
b = float(input("Initial balance: "))
n = int(input("Number of transactions: "))
l = float(input("Withdrawal limit: "))
d = w = s = f = 0
for i in range(n):
    c = int(input("1.Deposit 2.Withdraw: "))
    a = float(input("Enter amount: "))
    if a <= 0 or c not in [1, 2]:
        f += 1
    elif c == 1:
        b += a
        d += a
        s += 1
    elif a > l or a > b:
        f += 1
    else:
        b -= a
        w += a
        s += 1
print("Deposits:", d, "Withdrawals:", w)
print("Successful:", s, "Failed:", f)
print("Final Balance:", round(b, 2))
```

4. Output (Sample Output)

Initial balance: 5000

Number of transactions: 4

Withdrawal limit: 2000

1.Deposit 2.Withdraw: 1

Enter amount: 1000

1.Deposit 2.Withdraw: 2

Enter amount: 1500

1.Deposit 2.Withdraw: 2

Enter amount: 4000

1.Deposit 2.Withdraw: 1

Enter amount: 500

Deposits: 1500.0 Withdrawals: 1500.0

Successful: 3 Failed: 1

Final Balance: 5000.0

5. Conclusion

The Python program successfully manages bank deposits and withdrawals by validating transactions and checking withdrawal limits and available balances. It maintains the account balance and provides a summary of successful and failed transactions, helping users monitor their banking activities.

Project-4

1. Problem Statement

Develop a Python program to manage the inventory of multiple products by calculating the expected closing stock based on opening stock, incoming quantities, and dispatched quantities. The program should compare the expected stock with the actual closing stock, identify any discrepancies, and calculate the quantity required to replenish stock to the minimum stock level.

2. Objectives

1. To accept the number of products and their inventory details.
2. To calculate the expected closing stock for each product.
3. To compare the expected closing stock with the actual closing stock.
4. To identify and display any stock discrepancies.
5. To calculate the replenishment quantity required to reach the minimum stock level.
6. To display the inventory details for each product.

3. Assumptions

1. The number of products and all stock quantities are entered as integers.
2. Opening stock, incoming quantity, dispatched quantity, actual closing stock, and minimum stock are assumed to be non-negative.
3. Expected closing stock is calculated by subtracting dispatched quantity from the sum of opening stock and incoming quantity.
4. A discrepancy is calculated as actual closing stock minus expected closing stock.
5. If actual closing stock is equal to expected closing stock, the program displays "Stock matches."
6. Replenishment is required only when actual closing stock is below the minimum stock level.

Algorithm

1. Start.
2. Enter the number of products.
3. Enter the stock details for each product.
4. Check whether the stock details are valid.
5. Calculate the expected closing stock.
6. Compare the expected stock with the actual closing stock.
7. Display any stock discrepancy.
8. Calculate the replenishment required.
9. Display the results.
10. Repeat for all products.
11. Stop.

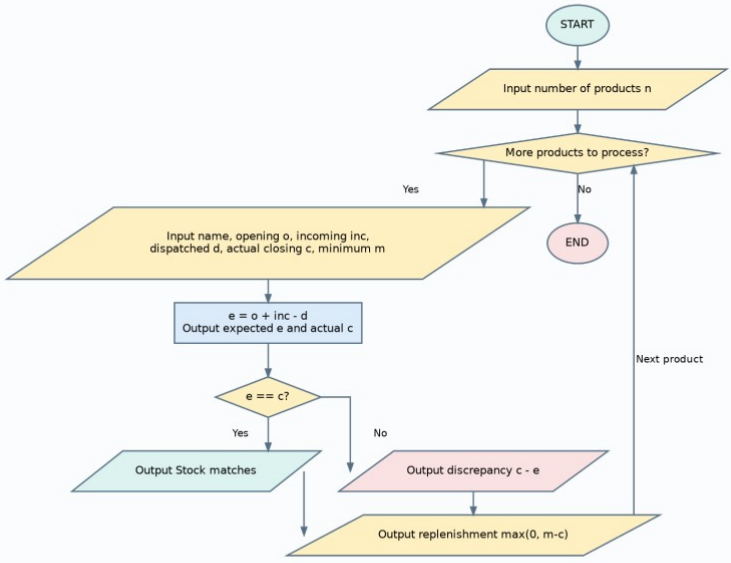
Product Stock Reconciliation

Source: pp-4.py

1. Pseudocode

```
START
INPUT number of products n
FOR each product i from 0 to n-1:
INPUT name, opening o, incoming inc,
dispatched d, actual closing c,
minimum stock m
e = o + inc - d
OUTPUT expected stock e and actual c
IF e == c:
OUTPUT 'Stock matches'
ELSE:
OUTPUT discrepancy c - e
OUTPUT replenishment max(0, m-c)
END FOR
END
```

2. Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Code

```
n = int(input("Number of products: "))
for i in range(n):
    name = input("Product name: ")
    o = int(input("Opening stock: "))
    inc = int(input("Incoming quantity: "))
    d = int(input("Dispatched quantity: "))
    c = int(input("Actual closing stock: "))
    m = int(input("Minimum stock: "))
    e = o + inc - d
    print("Expected stock:", e)
    print("Actual stock:", c)
    if e == c:
        print("Stock matches")
    else:
        print("Discrepancy:", c - e)
    print("Replenishment:", max(0, m - c))
```

4. Output (Sample Output)

Number of products: 2

Product name: Notebook

Opening stock: 100

Incoming quantity: 50

Dispatched quantity: 80

Actual closing stock: 65

Minimum stock: 40

Expected stock: 70

Actual stock: 65

Discrepancy: -5

Replenishment: 0

Product name: Pen

Opening stock: 30

Incoming quantity: 20

Dispatched quantity: 35

Actual closing stock: 10

Minimum stock: 25

Expected stock: 15

Actual stock: 10

Discrepancy: -5

Replenishment: 15

5. Conclusion

The Python program helps manage inventory by calculating expected stock, comparing it with actual stock, identifying discrepancies, and determining replenishment requirements. It supports effective stock monitoring and helps maintain sufficient inventory levels.

Project-5

1. Problem Statement

Develop a Python program to calculate parking duration and parking fees for different types of vehicles, including bikes, cars, and buses. The program should accept entry and exit times, calculate the parking duration in hours, apply the appropriate hourly rate and maximum fee limit, and include additional charges when applicable.

2. Objectives

1. To accept the vehicle's entry time, exit time, and vehicle type.
2. To calculate the parking duration in hours using the given times.
3. To apply different hourly parking rates for bikes, cars, and buses.
4. To ensure that the parking fee does not exceed the maximum fee limit for the selected vehicle.
5. To apply additional charges for parking that extends into the next day or exceeds 24 hours, according to the program's conditions.
6. To display the parking duration and final parking fee.

3. Assumptions

1. Entry and exit times are entered in HH:MM format using a 24-hour clock.
2. The vehicle type must be bike, car, or bus.
3. The hourly rates are Rs. 10 for bikes, Rs. 20 for cars, and Rs. 40 for buses.
4. The maximum parking fees are Rs. 100 for bikes, Rs. 200 for cars, and Rs. 400 for buses.
5. A minimum of one parking hour is charged, and any partial hour is rounded up.
6. If the exit time is earlier than the entry time, the program assumes that the vehicle exited the following day.

7. An additional charge of Rs. 50 is applied when the calculated exit time in minutes is at least 1440.
8. An additional charge of Rs. 100 is applied when the calculated parking duration exceeds 24 hours.
9. The program assumes that the entered times are valid.

Algorithm

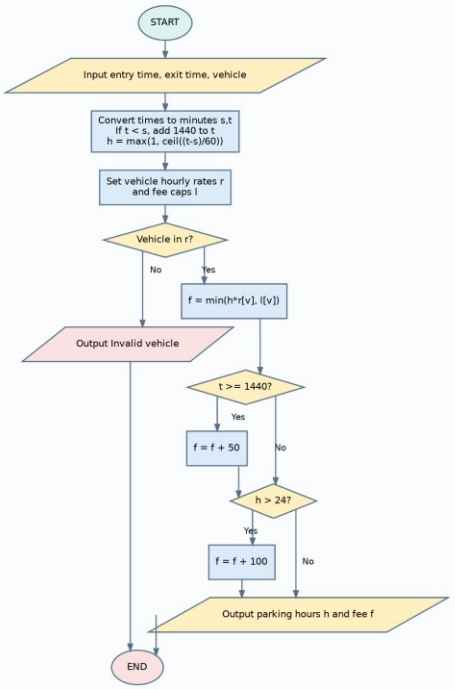
1. Start.
2. Enter the entry time, exit time, and vehicle type.
3. Check whether the time and vehicle type are valid.
4. Calculate the parking duration in hours.
5. Calculate the parking fee based on the vehicle type.
6. Apply the daily fee limit and additional charges, if applicable.
7. Display the parking duration, vehicle type, and total fee.
8. Stop.

Parking Fee Calculator — Source: pp-5.py

1. Pseudocode

```
START
INPUT entry time e, exit time x, vehicle v
Convert each time to minutes: s, t
IF t < s: t = t + 1440
h = max(1, ceil((t-s)/60))
Set hourly rates r and fee caps l:
bike: 10, 100; car: 20, 200;
bus: 40, 400
IF v is not in r:
OUTPUT 'Invalid vehicle'
ELSE:
f = min(h * r[v], l[v])
IF t >= 1440: f = f + 50
IF h > 24: f = f + 100
OUTPUT parking hours h and fee f
END
```

2. Flowchart



Code

```
e = input("Entry time (HH:MM): ").split(":")
x = input("Exit time (HH:MM): ").split(":")
v = input("Vehicle (bike/car/bus): ").lower()
s = int(e[0])*60 + int(e[1])
t = int(x[0])*60 + int(x[1])
if t < s: t += 1440
h = max(1, (t-s+59)//60)
r = {"bike":10, "car":20, "bus":40}
l = {"bike":100, "car":200, "bus":400}
if v not in r: print("Invalid vehicle")
else:
    f = min(h*r[v], l[v])
    if t >= 1440: f += 50
    if h > 24: f += 100
    print("Parking hours:", h)
    print("Parking fee: Rs.", f)
```

4. Output (Sample Output)

Entry time (HH:MM): 09:00

Exit time (HH:MM): 13:30

Vehicle (bike/car/bus): car

Parking hours: 5

Parking fee: Rs. 100

5. Conclusion

The Python program calculates parking duration and fees based on entry time, exit time, and vehicle type. It applies the appropriate hourly rates, maximum fee limits, and additional charges according to the specified conditions. This helps automate parking fee calculation for different vehicles.

Project-6

1. Problem Statement

Develop a Python program to monitor the water level in a tank over a specified number of days. The program should accept the tank capacity, initial water level, and target water level. For each day, it should calculate the updated water level based on water inflow and outflow, ensure that the water level remains within the tank's capacity, and determine whether the target level has been reached.

2. Objectives

1. To accept the tank capacity, initial water level, target level, and number of days.
2. To validate the initial tank details.
3. To calculate the daily water level using inflow and outflow values.

4. To prevent the water level from exceeding the tank capacity or falling below zero.
5. To display the water level at the end of each day.
6. To identify and display the first day on which the target water level is reached.
7. To display a message if the target level is not reached during the specified period.

3. Assumptions

1. Tank capacity, water levels, inflow, and outflow are entered as floating-point numbers.
2. The number of days is entered as an integer greater than zero.
3. Tank capacity must be greater than zero.
4. The initial water level must be between zero and the tank capacity.
5. Daily inflow and outflow values are assumed to be non-negative.
6. The water level is calculated by adding inflow and subtracting outflow from the current level.
7. The water level cannot exceed the tank capacity or fall below zero.
8. The target is considered reached when the water level is greater than or equal to the target level.

Algorithm

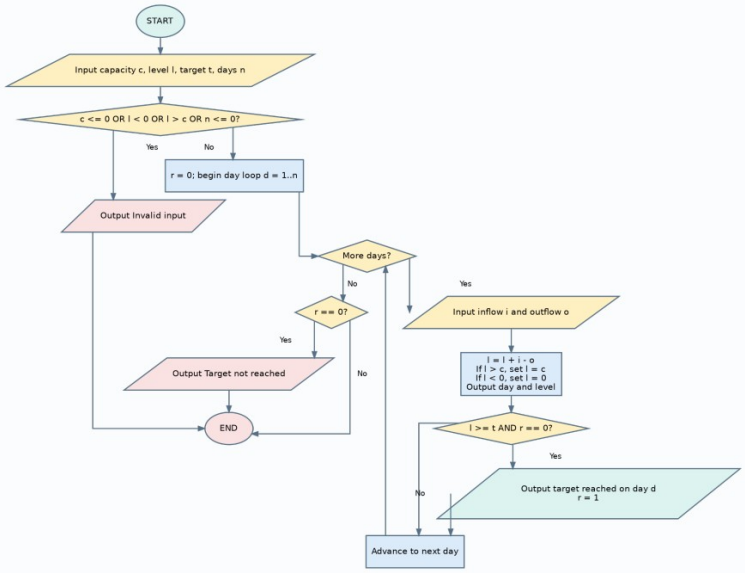
1. Start.
2. Enter the tank capacity, initial water level, target level, and number of days.
3. Check whether the input values are valid.
4. Enter the daily inflow and outflow.
5. Calculate the current water level.
6. Check for overflow or water shortage.
7. Display the water level for each day.
8. Check when the target water level is first reached.
9. If the target is not reached, display a message.
10. Stop.

Daily Water Tank Level Tracker — Source: pp-6.py

1. Pseudocode

```
START
INPUT capacity c, initial level l, target t, days n
IF c <= 0 OR l < 0 OR l > c OR n <= 0:
OUTPUT 'Invalid input'
ELSE:
r = 0
FOR d from 1 through n:
INPUT inflow i and outflow o
l = l + i - o
IF l > c: l = c
IF l < 0: l = 0
OUTPUT day d and water level l
IF l >= t AND r == 0:
OUTPUT target reached on day d
r = 1
IF r == 0: OUTPUT target not reached
END
```

2. Flowchart



Code

```
c = float(input("Tank capacity: "))
l = float(input("Initial water level: "))
t = float(input("Target level: "))
n = int(input("Number of days: "))
if c <= 0 or l < 0 or l > c or n <= 0:
    print("Invalid input")
else:
    r = 0
    for d in range(1, n+1):
        i = float(input("Inflow: "))
        o = float(input("Outflow: "))
        l = l + i - o
        if l > c: l = c
        if l < 0: l = 0
        print("Day", d, "Water level:", l)
        if l >= t and r == 0:
            print("Target reached on day", d)
            r = 1
    if r == 0: print("Target not reached")
```

4. Output (Sample Output)

Tank capacity: 1000

Initial water level: 400

Target level: 700

Number of days: 3

Inflow: 200

Outflow: 50

Day 1 Water level: 550.0

Inflow: 250

Outflow: 50

Day 2 Water level: 750.0

Target reached on day 2

Inflow: 100

Outflow: 80

Day 3 Water level: 770.0

5. Conclusion

The Python program monitors daily changes in a water tank by calculating inflow and outflow and maintaining the water level within the tank's capacity. It identifies when the target water level is first reached and displays the daily water levels, helping users track water availability effectively.

Project-7

1. Problem Statement

Develop a Python program to calculate the parking duration and parking fee for bikes, cars, and buses based on their entry and exit times. The program should calculate the parking duration in hours, apply the appropriate hourly rate and daily maximum fee, and include additional charges when applicable. It should also identify invalid vehicle types.

2. Objectives

1. To accept the vehicle's entry time, exit time, and vehicle type.
2. To calculate the total parking duration in hours, rounding up partial hours.
3. To apply different hourly parking rates for bikes, cars, and buses.
4. To calculate the number of parking days required for the duration.
5. To apply the maximum parking fee for each vehicle type based on the number of parking days.
6. To add additional charges according to the program's conditions.
7. To display the total parking duration and final parking fee.

3. Assumptions

1. Entry and exit times are entered in HH:MM format using a 24-hour clock.
2. The vehicle type must be bike, car, or bus.
3. The hourly rates are Rs. 10 for bikes, Rs. 20 for cars, and Rs. 40 for buses.
4. The daily maximum fees are Rs. 100 for bikes, Rs. 200 for cars, and Rs. 400 for buses.
5. A minimum of one parking hour is charged, and partial hours are rounded up.
6. If the exit time is earlier than the entry time, the program assumes that the vehicle exited the following day.
7. The number of parking days is calculated by rounding up the parking duration to complete 24-hour periods, with a minimum of one day.
8. An additional charge of Rs. 50 is applied when the calculated exit time in minutes is at least 1440.
9. An additional charge of Rs. 100 is applied when the parking duration exceeds 24 hours.

10. The program assumes that the entered times are valid.

Algorithm

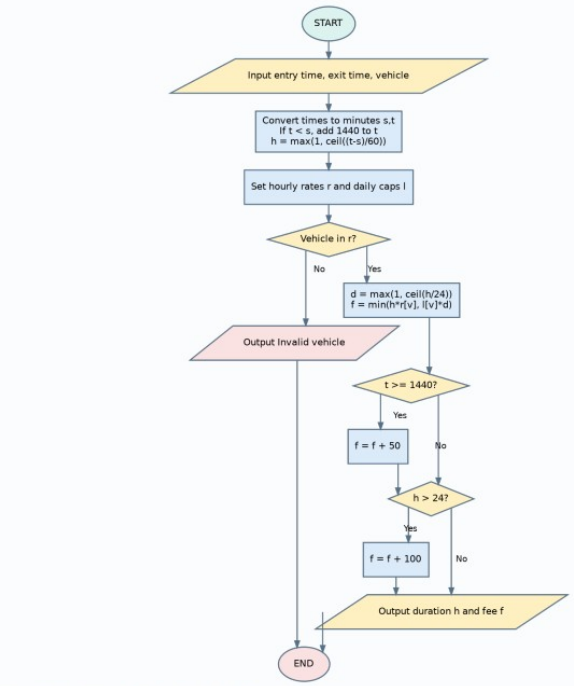
1. Start.
2. Enter the vehicle entry time, exit time, and vehicle type.
3. Check whether the time format and vehicle type are valid.
4. Calculate the parking duration in minutes.
5. Convert the duration into hours, rounding up to the next full hour.
6. Select the hourly rate and daily fee limit based on the vehicle type.
7. Calculate the parking fee using the hourly rate and daily limit.
8. Add an extra fee if the vehicle exits on the next day.
9. Add an extra fee if the parking duration exceeds 24 hours.
10. Display the parking duration, vehicle type, and total parking fee.
11. Stop.

Parking Fee Calculator (Daily Cap) Source: pp-7.py

1. Pseudocode

```
START
INPUT entry time e, exit time x, vehicle v
Convert times to minutes: s, t
IF t < s: t = t + 1440
h = max(1, ceil((t-s)/60))
Set hourly rates r and daily caps l:
bike: 10, 100; car: 20, 200;
bus: 40, 400
IF v is not in r:
OUTPUT 'Invalid vehicle'
ELSE:
d = max(1, ceil(h/24))
f = min(h*r[v], l[v]*d)
IF t >= 1440: f = f + 50
IF h > 24: f = f + 100
OUTPUT duration h and fee f
END
```

2. Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Code

```
e = input("Entry time (HH:MM): ").split(":")
x = input("Exit time (HH:MM): ").split(":")
v = input("Vehicle (bike/car/bus): ").lower()
s = int(e[0])*60 + int(e[1])
t = int(x[0])*60 + int(x[1])
if t < s: t += 1440
h = max(1, (t-s+59)//60)
r = {"bike":10, "car":20, "bus":40}
l = {"bike":100, "car":200, "bus":400}
if v not in r: print("Invalid vehicle")
else:
    d = max(1, (h+23)//24)
    f = min(h*r[v], l[v]*d)
    if t >= 1440: f += 50
    if h > 24: f += 100
    print("Duration:", h, "hours")
    print("Parking fee: Rs.", f)
```

4. Output (Sample Output)

Entry time (HH:MM): 09:00

Exit time (HH:MM): 13:30

Vehicle (bike/car/bus): car

Duration: 5 hours

Parking fee: Rs. 100

5. Conclusion

The Python program calculates parking duration and fees for different vehicle types using hourly rates and daily maximum fee limits. It also applies additional charges based on the parking duration and exit time. This program helps automate parking fee calculation and provides a systematic way to manage vehicle parking charges.

Project-8

1. Problem Statement

Develop a Python program to calculate the fuel requirements and total fuel cost for multiple vehicles based on distance, mileage, fuel price, and reserve fuel. The program should compare the calculated costs with a given fuel budget and identify the most economical vehicle that fits within the budget.

2. Objectives

1. To accept the number of vehicles and the available fuel budget.
2. To collect the distance, mileage, fuel price, and reserve fuel details for each vehicle.
3. To calculate the total fuel required for each vehicle.
4. To calculate and display the total fuel cost for each vehicle.
5. To compare each vehicle's fuel cost with the available budget.
6. To identify the vehicle with the lowest fuel cost among those within the budget.
7. To display a message if no vehicle meets the budget requirement.

3. Assumptions

1. The number of vehicles is entered as an integer.
2. The fuel budget, distance, mileage, fuel price, and reserve fuel are entered as floating-point numbers.
3. Distance, mileage, fuel price, and reserve fuel are assumed to be valid positive values, with mileage greater than zero.
4. Fuel required is calculated by dividing distance by mileage and adding the reserve fuel.
5. Total fuel cost is calculated by multiplying the fuel required by the fuel price per unit.
6. Only vehicles whose total fuel cost is less than or equal to the budget are considered.
7. If multiple vehicles meet the budget, the vehicle with the lowest fuel cost is selected.
8. If no vehicle is within the budget, the program displays an appropriate message.

Algorithm

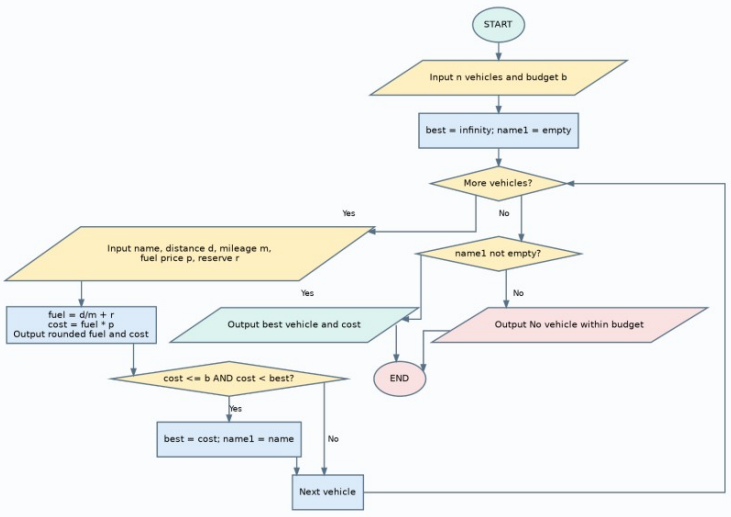
1. Start.
2. Enter the number of vehicles and the fuel budget.
3. Enter each vehicle's name, journey distance, mileage, fuel price, and reserve fuel.
4. Check whether the vehicle details are valid.
5. Calculate the fuel required for the journey.
6. Add the reserve fuel to the required fuel.
7. Calculate the total fuel cost.
8. Display the fuel requirement and journey cost for each vehicle.
9. Compare the cost with the budget and identify the lowest-cost feasible vehicle.
10. Display the best vehicle and its total fuel cost, or display a message if no vehicle fits the budget.
11. Stop.

Fuel Cost & Best Vehicle Selection Source: pp-8.py

1. Pseudocode

```
START
INPUT number of vehicles n and budget b
best = infinity; name1 = empty
FOR each vehicle i from 0 to n-1:
INPUT name, distance d, mileage m,
fuel price p, reserve fuel r
fuel = d/m + r
cost = fuel * p
OUTPUT rounded fuel and cost
IF cost <= b AND cost < best:
best = cost
name1 = name
END FOR
IF name1 is not empty:
OUTPUT best vehicle name1 and best cost
ELSE:
OUTPUT 'No vehicle within budget'
END
```

2. Flowchart



Code

```
n = int(input("Number of vehicles: "))
b = float(input("Fuel budget: "))
best = float("inf")
name1 = ""
for i in range(n):
    name = input("Vehicle name: ")
    d = float(input("Distance: "))
    m = float(input("Mileage: "))
    p = float(input("Fuel price: "))
    r = float(input("Reserve fuel: "))
    fuel = d/m + r
    cost = fuel*p
    print("Fuel required:", round(fuel, 2))
    print("Total cost: Rs.", round(cost, 2))
    if cost <= b and cost < best:
        best = cost
        name1 = name
if name1:
    print("Best vehicle:", name1, "Cost: Rs.", round(best, 2))
else:
    print("No vehicle within budget")
```

4. Output (Sample Output)

Number of vehicles: 3

Fuel budget: 1500

Vehicle name: Car

Distance: 300

Mileage: 15

Fuel price: 100

Reserve fuel: 2

Fuel required: 22.0

Total cost: Rs. 2200.0

Vehicle name: Bike

Distance: 300

Mileage: 40

Fuel price: 100

Reserve fuel: 1

Fuel required: 8.5

Total cost: Rs. 850.0

Vehicle name: Bus

Distance: 300

Mileage: 5

Fuel price: 100

Reserve fuel: 5

Fuel required: 65.0

Total cost: Rs. 6500.0

Best vehicle: Bike Cost: Rs. 850.0

5. Conclusion

The Python program calculates fuel requirements and fuel costs for multiple vehicles and compares them with the available budget. It identifies the most economical vehicle that satisfies the budget constraint, helping users make cost-effective travel decisions.

Project-9

1. Problem Statement

Develop a Python program to calculate the total cost of a shopping basket containing multiple products. The program should calculate the total price based on product prices and quantities, apply discounts according to the total purchase amount, and compare the final bill with the available shopping budget. If the final cost exceeds the budget, the program should display the amount by which the purchases exceed the budget.

2. Objectives

1. To accept the number of products and the available shopping budget.
2. To collect the name, price, and quantity of each product.
3. To calculate the total shopping cost before applying discounts.
4. To apply a 10% discount for purchases of Rs. 5,000 or more and a 5% discount for purchases of Rs. 2,000 or more but less than Rs. 5,000.
5. To calculate and display the discount amount and final bill.
6. To compare the final bill with the available budget.

7. To display the additional amount required if the final bill exceeds the budget.

3. Assumptions

1. The number of products and quantities are entered as integers.
2. The shopping budget and product prices are entered as floating-point numbers.
3. Product prices and quantities are assumed to be valid positive values.
4. The total cost is calculated by multiplying each product's price by its quantity and adding the amounts.
5. A 10% discount is applied when the total cost is Rs. 5,000 or more.
6. A 5% discount is applied when the total cost is Rs. 2,000 or more but less than Rs. 5,000.
7. No discount is applied when the total cost is less than Rs. 2,000.
8. The final bill is compared with the shopping budget to determine whether the purchase is within budget.

Algorithm

1. Start.
2. Enter the number of products and the shopping budget.
3. Enter each product's name, price, and quantity.
4. Check whether the product details are valid.
5. Calculate the amount for each product and the total shopping cost.
6. Apply a 10% discount if the total is at least ₹5,000, or a 5% discount if it is at least ₹2,000.
7. Calculate and display the total, discount, and final amount.
8. Compare the final amount with the budget.
9. If the amount exceeds the budget, calculate the excess and reduce product quantities.
10. Otherwise, display that the shopping is within budget.
11. Stop.

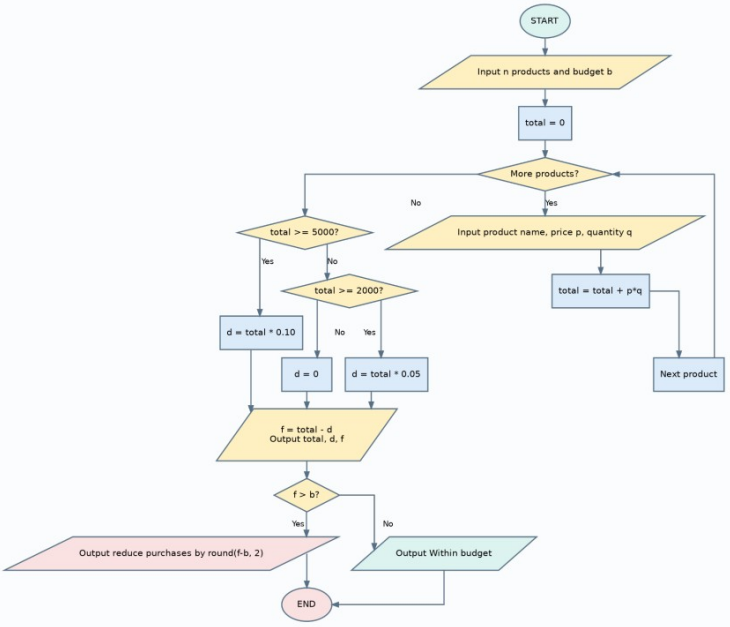
Shopping Budget & Discount Calculator

Source: pp-9.py

1. Pseudocode

```
START
INPUT number of products n and budget b
total = 0
FOR each product i from 0 to n-1:
INPUT product name, price p, quantity q
total = total + p * q
END FOR
IF total >= 5000: d = total * 0.10
ELSE IF total >= 2000: d = total * 0.05
ELSE: d = 0
f = total - d
OUTPUT total, discount d, final amount f
IF f > b:
OUTPUT amount to reduce: round(f-b, 2)
ELSE:
OUTPUT 'Within budget'
END
```

2. Flowchart



Code

```
n = int(input("Number of products: "))
b = float(input("Shopping budget: "))
total = 0
for i in range(n):
    name = input("Product name: ")
    p = float(input("Price: "))
    q = int(input("Quantity: "))
    total += p * q
if total >= 5000: d = total * 0.10
elif total >= 2000: d = total * 0.05
else: d = 0
f = total - d
print("Total:", total, "Discount:", d, "Final:", f)
if f > b: print("Reduce purchases by Rs.", round(f-b, 2))
else: print("Within budget")
```

4. Output (Sample Output)

Number of products: 3

Shopping budget: 3000

Product name: Shoes

Price: 1500

Quantity: 1

Product name: Shirt

Price: 800

Quantity: 2

Product name: Belt

Price: 400

Quantity: 1

Total: 3500.0 Discount: 350.0 Final: 3150.0

Reduce purchases by Rs. 150.0

5. Conclusion

The Python program calculates the total shopping cost, applies discounts based on purchase value, and determines whether the final bill fits within the available budget. If the budget is exceeded, it displays the additional amount required, helping users plan their purchases and manage their spending effectively.

Project-10

1. Problem Statement

Develop a Python program to evaluate university admission applicants based on their Mathematics, Science, and English marks, average marks, and attendance percentage. The program should determine whether each applicant is eligible for admission according to the specified criteria. Eligible applicants should be ranked in descending order of average marks, with attendance percentage used as the tie-breaker.

2. Objectives

1. To accept the number of applicants and their academic details.
2. To calculate the average marks of each applicant.

3. To check eligibility based on subject-wise marks, average marks, and attendance percentage.
4. To display the eligibility status of each applicant.
5. To store the details of eligible applicants.
6. To rank eligible applicants in descending order of average marks.
7. To use attendance percentage as the secondary ranking criterion when applicants have equal average marks.
8. To display the ranked list with each applicant's rank, name, average marks, and attendance percentage.

3. Assumptions

1. The number of applicants is entered as an integer.
2. Applicant names are entered as text.
3. Subject marks and attendance percentages are entered as floating-point numbers.
4. An applicant must score at least 50 marks in Mathematics, 50 in Science, and 40 in English.
5. The applicant's average marks must be at least 60, and attendance must be at least 75%.
6. Only applicants satisfying all the eligibility conditions are included in the ranked list.
7. Eligible applicants are ranked by average marks in descending order.
8. If two applicants have equal average marks, the applicant with higher attendance is ranked first.

Algorithm

1. Start.
2. Enter the number of applicants.
3. Enter each applicant's name, Mathematics marks, Science marks, English marks, and attendance percentage.
4. Calculate the average marks of each applicant.
5. Check the eligibility conditions for marks, aggregate, and attendance.
6. Display whether each applicant is eligible or not.
7. Store the details of eligible applicants.
8. Sort eligible applicants by aggregate marks in descending order, then attendance in descending order, and finally name in alphabetical order.
9. Display the ranked list of eligible applicants.
10. If no applicants are eligible, display a suitable message.
11. Stop.

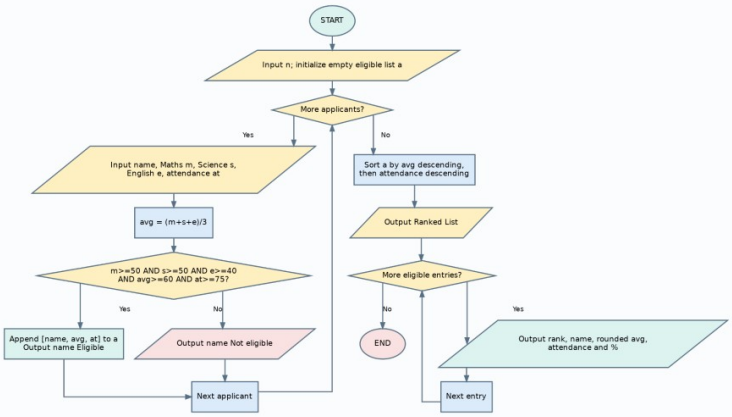
Applicant Eligibility & Ranking

Source: pp-10.py

1. Pseudocode

```
START
INPUT number of applicants n; a = empty list
FOR each applicant i from 0 to n-1:
INPUT name, Maths m, Science s,
English e, attendance at
avg = (m + s + e) / 3
IF m >= 50 AND s >= 50 AND e >= 40
AND avg >= 60 AND at >= 75:
append [name, avg, at] to a
OUTPUT name, 'Eligible'
ELSE:
OUTPUT name, 'Not eligible'
END FOR
Sort a by avg descending, then attendance
descending
OUTPUT 'Ranked List'
FOR each entry i in sorted a:
OUTPUT rank i+1, name, rounded avg,
attendance and %
END
```

2. Flowchart



Code

```
n = int(input("Number of applicants: "))
a = []
for i in range(n):
    name = input("Name: ")
    m = float(input("Maths marks: "))
    s = float(input("Science marks: "))
    e = float(input("English marks: "))
    at = float(input("Attendance: "))
    avg = (m+s+e)/3
    if m>=50 and s>=50 and e>=40 and avg>=60 and at>=75:
        a.append([name, avg, at])
        print(name, "Eligible")
    else: print(name, "Not eligible")
print("Ranked List:")
for i in range(len(a)):
    print(i+1, a[i][0], round(a[i][1], 2), a[i][2], "%")
```

4. Output (Sample Output)

Number of applicants: 3

Name: Ravi

Maths marks: 80

Science marks: 75

English marks: 70

Attendance: 90

Ravi Eligible

Name: Priya

Maths marks: 65

Science marks: 70

English marks: 60

Attendance: 85

Priya Eligible

Name: Arun

Maths marks: 45

Science marks: 80

English marks: 75

Attendance: 90

Arun Not eligible

Ranked List:

1 Ravi 75.0 90.0 %

2 Priya 65.0 85.0 %

5. Conclusion

The Python program evaluates university admission applicants using subject-wise marks, average marks, and attendance requirements. It identifies eligible applicants and ranks them according to academic performance, using attendance as a tie-breaker. This program helps simplify the admission evaluation and ranking process.